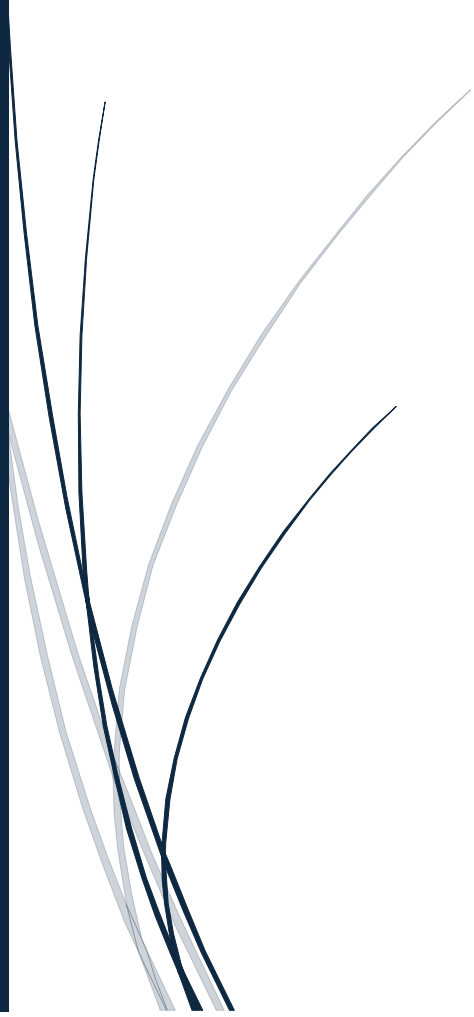
8/11/2024

Technical Report

Fundamentals of Data Science



Authur: MBKX3

Contents

Technical Summary 2
Introduction 2
Description 3
    Dataset Overview and Preprocessing 4
    Analysis of Formation Energy and Cluster Size Distribution 4
Data Analysis 5
    Analysis of Cluster Stability and Energy Distribution Across Sizes and Element 5
    Analysis of Correlations Between Physical Properties 6
    Principle Component Analysis 8
    Further Analysis of Transition Metals 10
Conclusion 11
Reference 11
Appendix 11
    Python code for creating plots 11

Technical Summary

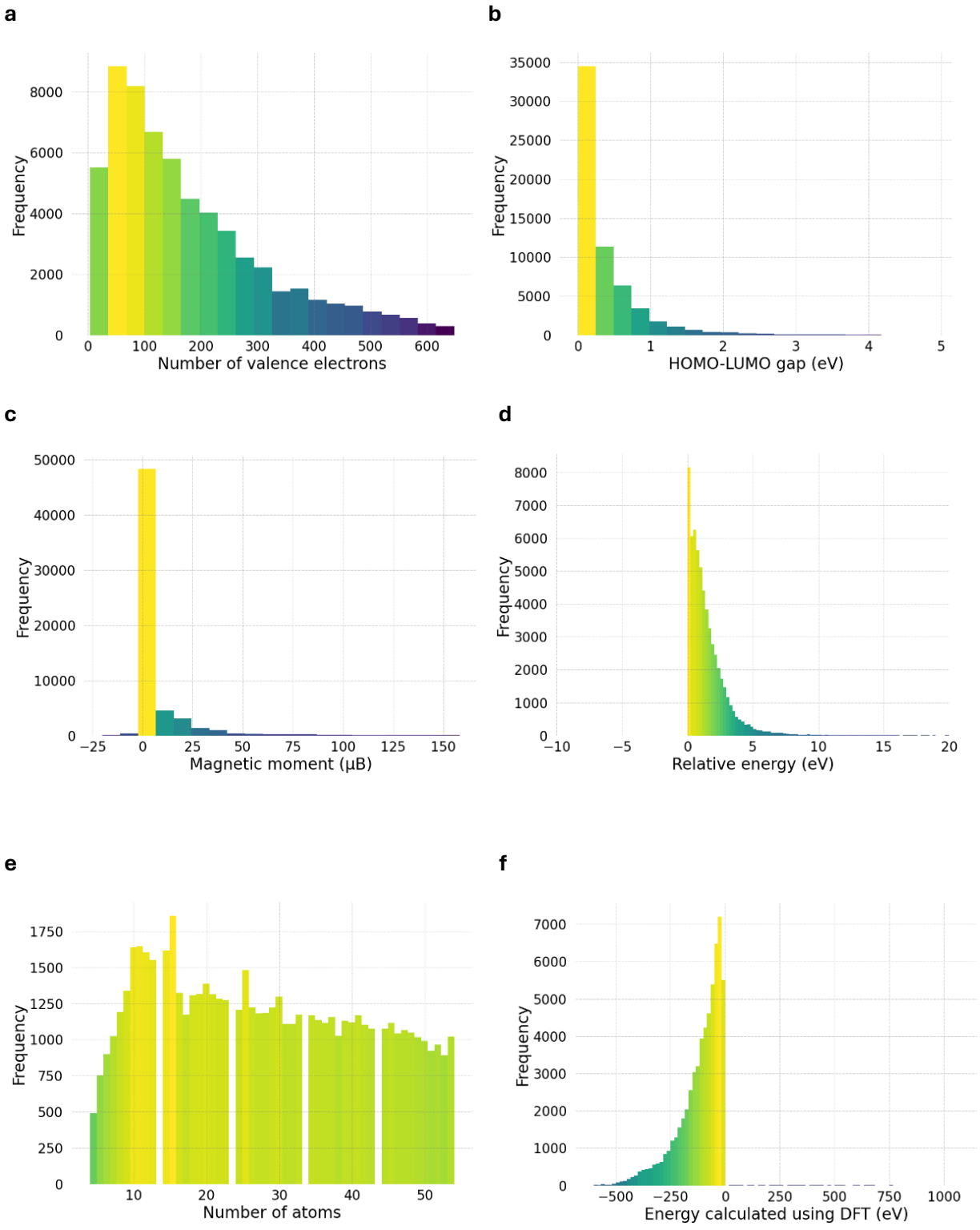
This report analyzes the Quantum Cluster Database using histograms, violin plots, heatmaps, scatter matrices, and PCA biplots. The findings serve as guidance for selecting stable nanoclusters from alkali metals, alkaline earth metals, transition metals, post-transition metals, and metalloids for research purposes. Additionally, the analysis reveals unusual behavior in Y nanoclusters.

Introduction

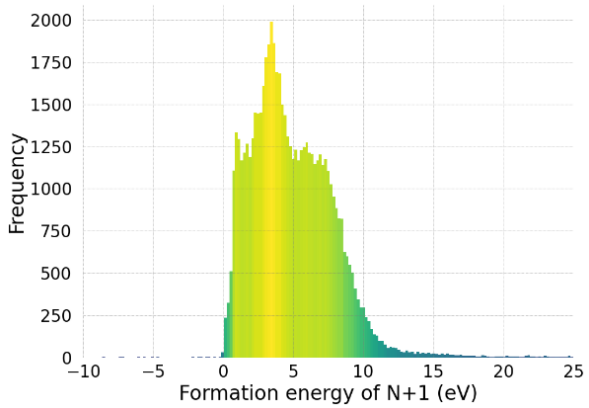
Nanocluster's defined chemical structures make it useful for studying nanomaterial's physical properties[1] .Thus, researchers have focused on finding stable nanoclusters to be experimented with and analyzed. The data chosen for this report is the Quantum Cluster Database developed by Manna et al. at Johns Hopkins University [1]. This dataset describes the physical properties of nanoclusters of different structures and element. The aim of this analysis is to classify nanoclusters and visualize the stability of nanoclusters belong to different families of the periodic table.

Description

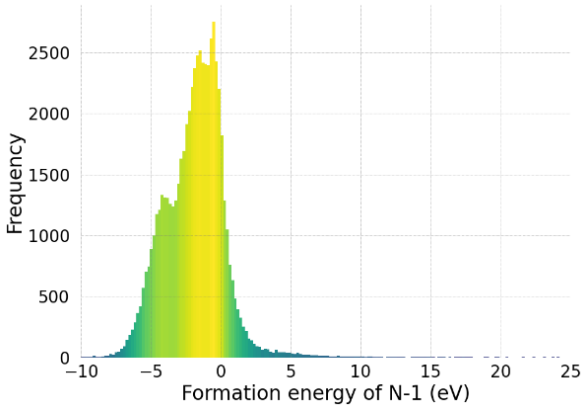
Fig .1 | Overview of nanocluster database. **a** number of valance electrons, **b** HOMO-LUMO gap, **c** magnetic moment , **d** relative energy, **e** number of atoms, **f** energy calculated using Density Field Theory (DFT), **g** formation energy of cluster with N atoms to N-1 atoms, **h** formation energy of cluster with N atoms to N-1 atoms, **I** element symbol.



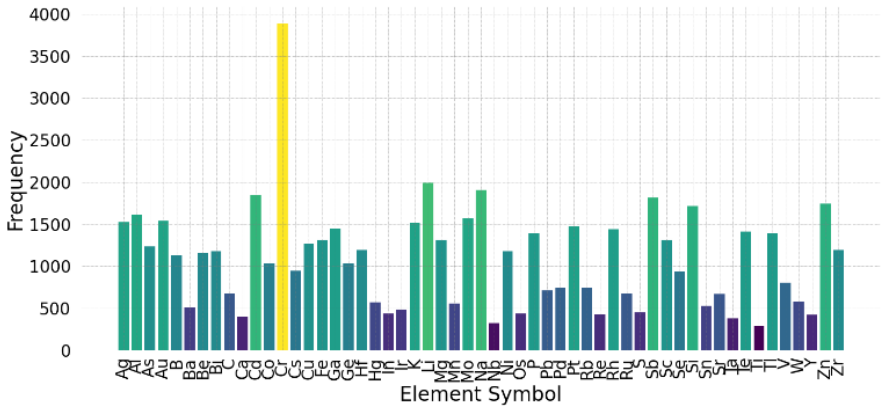
g



h



I



Dataset Overview and Preprocessing

This dataset contains physical properties of 63,015 nanoclusters across 55 elements[1]. **Fig.1** displays all nine numerical parameters included in the dataset. Definitions for frequently misunderstood parameters are provided:

1. Relative energy: energy compared to the lowest-energy structure of the same element and size[1]
2. Energy N-1 and N+1: formation energy of cluster with N atoms to N-1 or N+1 atoms
3. Energy calculated using DFT: represents total energy of cluster.

All clusters with missing values are deleted, reducing the number of clusters to 60,553. Since the number of atoms in a cluster is a pre-set parameter to defined structure of cluster, it is not considered as a principal component for PCA but is included in other analysis.

Analysis of Formation Energy and Cluster Size Distribution

Most nanoclusters have negative formation energy when losing an atom (**Fig.1h**), while they exhibit positive formation energy when adding an atom (**Fig.1g**), suggesting unstable nanoclusters are more prone to breaking down into smaller molecules than aggregating into larger nanoparticles. All clusters have negative energy calculated using DFT (**Fig.1f**) shows that clusters are formed spontaneously.

Data Analysis

Analysis of Cluster Stability and Energy Distribution Across Sizes and Element

Fig.2 | Distribution of Energy (DFT) by Number of Atoms (n_atoms)

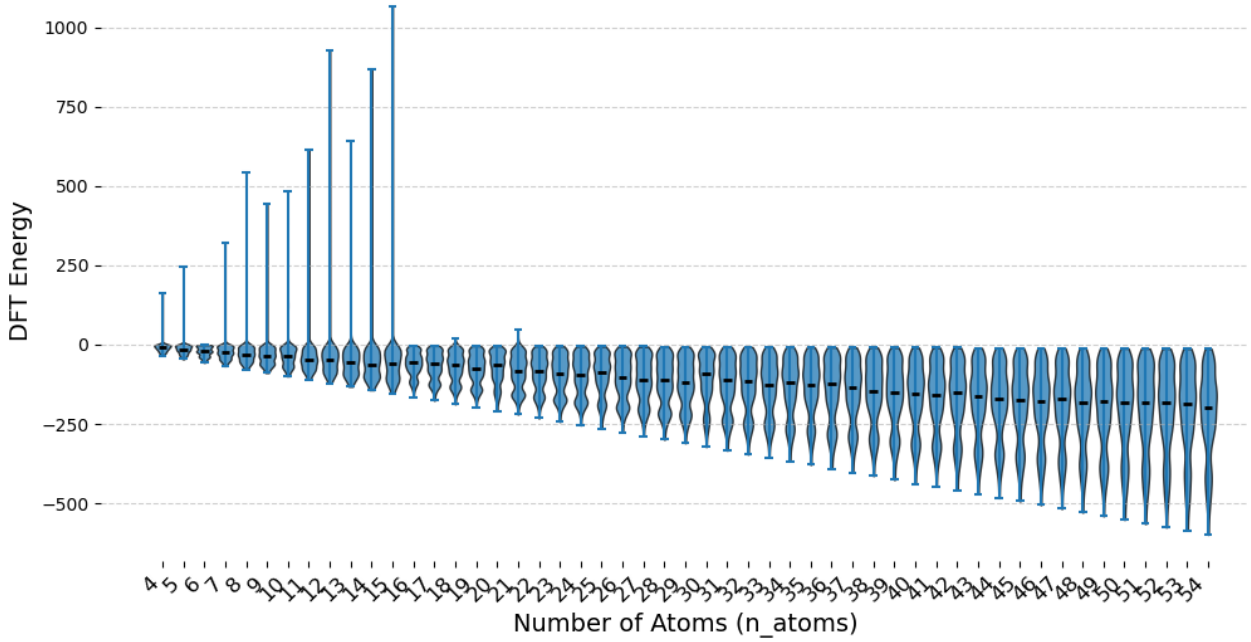


Fig.3 | Energy Distribution for All Elements

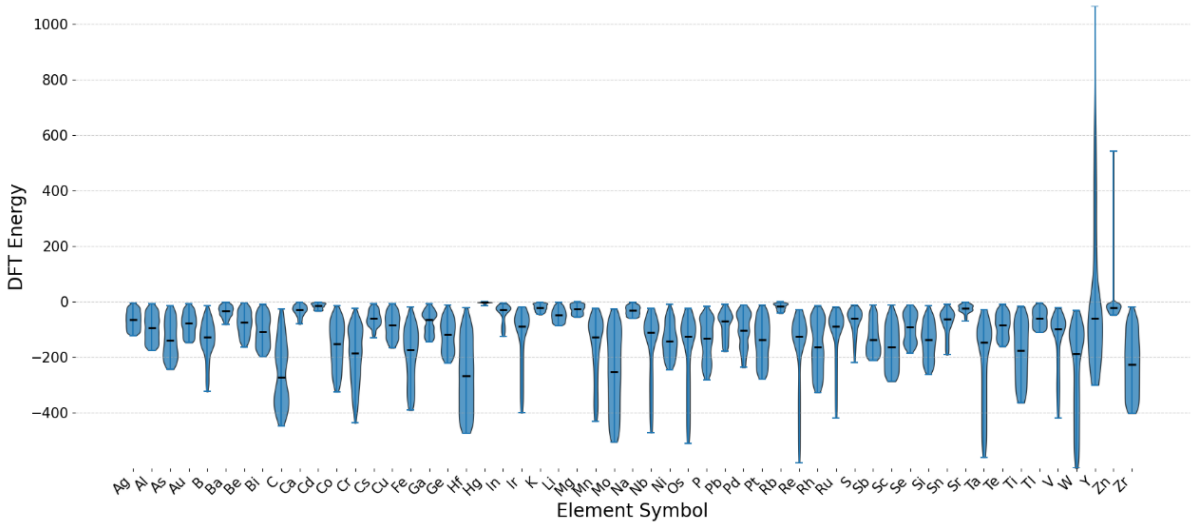


Fig. 2 shows that when the number of atoms increases, the median total energy (energy DFT) shifts downward, and the probability of a lower total energy increases. This means nanoclusters with a higher number of atoms are more stable. For clusters composed of 4 to 15 atoms, the long line above the upper quartile represents outliers, indicating clusters are not only unstable within this atom range but the number of atoms alone is insufficient to explain total energy of cluster. Other factors, such as element type, could also play a role.

Fig. 3 further supports the finding that atom energy varies across elements, with nanoclusters composed of Yttrium (Y) being the most unstable, followed by Zinc. Since the cause of this large range is unclear, no outliers were removed.

Analysis of Correlations Between Physical Properties

Fig.4 | Correlation Heat Map of Properties

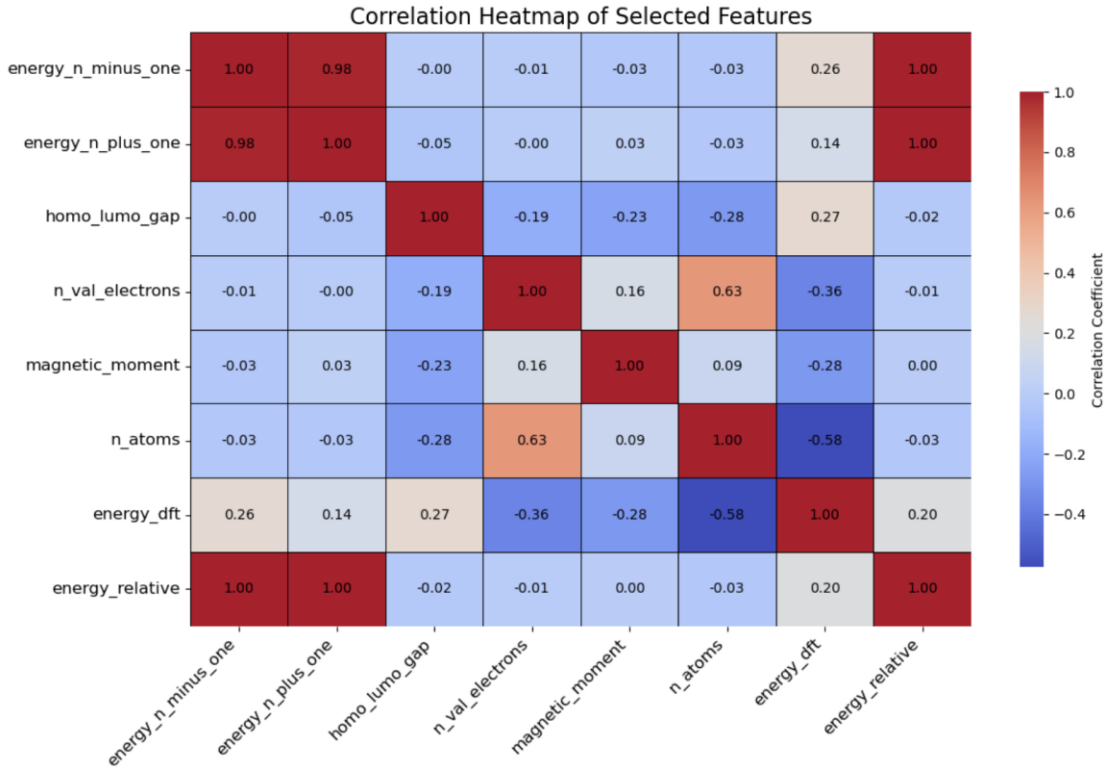
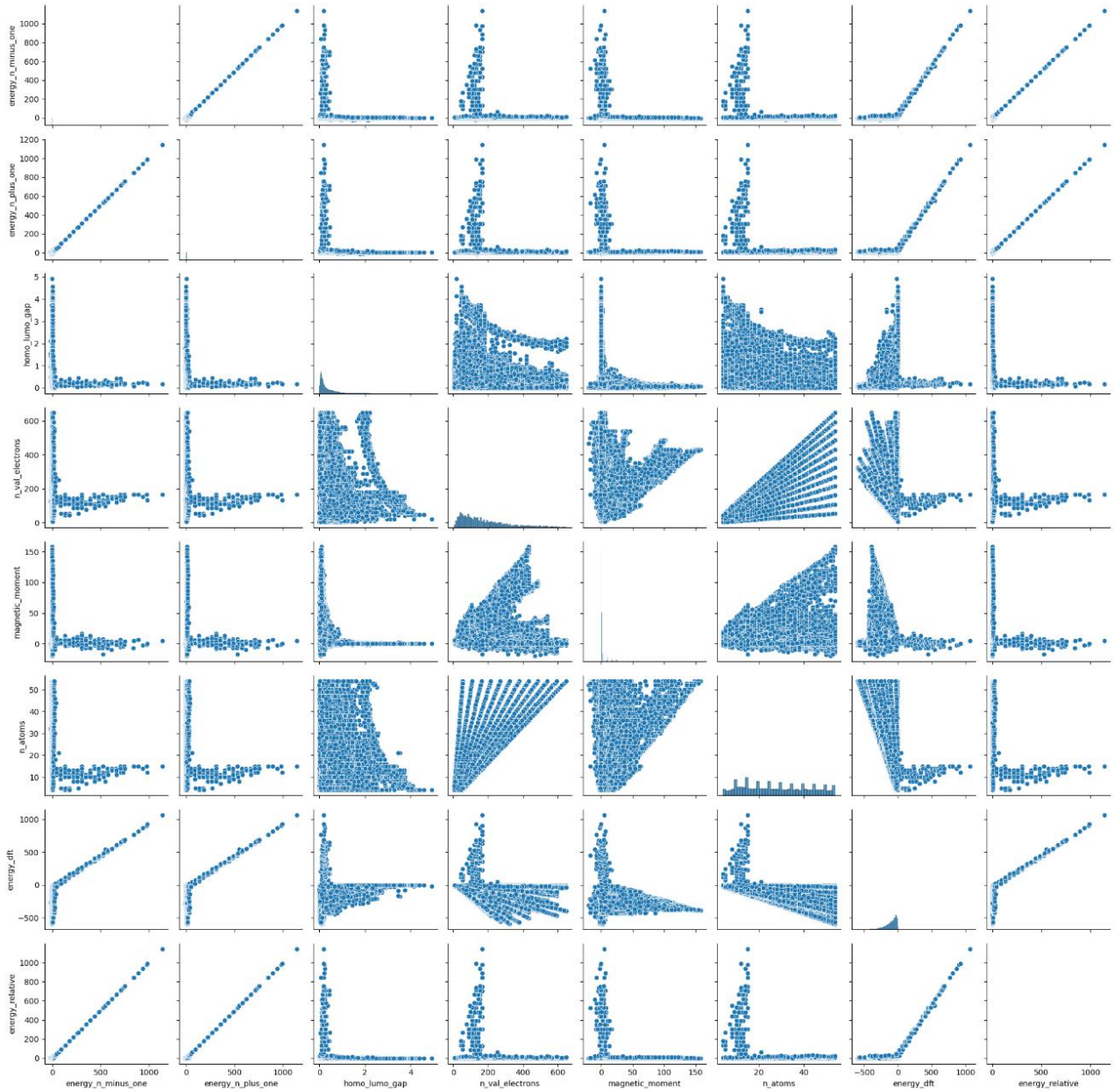


Fig.4 shows a strong positive correlation between relative energy and formation energies. This is reasonable because formation energy and relative energy all describe the tendency of nanocluster to assemble or dissemble, so if the relative energy is high, the energy required to form a smaller or larger nanocluster will also be high, and vice versa.

Fig.5 | Scatter Matrix of Properties



A large HOMO-LUMO gap lead to low reactivity and high stability, suggesting that clusters with larger gaps should have higher formation energy since they cannot easily react and resemble [2]. However, **Fig.5** reveals clusters either exhibit both low HOMO-LUMO gaps and low formation energy, or they have one property high while the other is low. No clusters display both a large HOMO-LUMO gap and high energy. This discrepancy may stem from the fact that these clusters are computationally modeled or formed in isolated environments, where reaction with other clusters or molecules is unlikely. As a result, an increase in the HOMO-LUMO gap may not directly impact cluster energy levels; instead, it may primarily influence the wavelength of cluster luminescence.

Principle Component Analysis

Fig.6 | Scree Plot

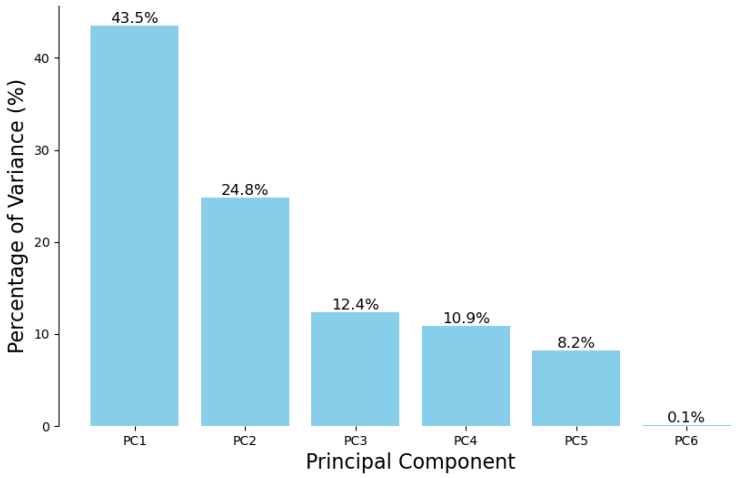
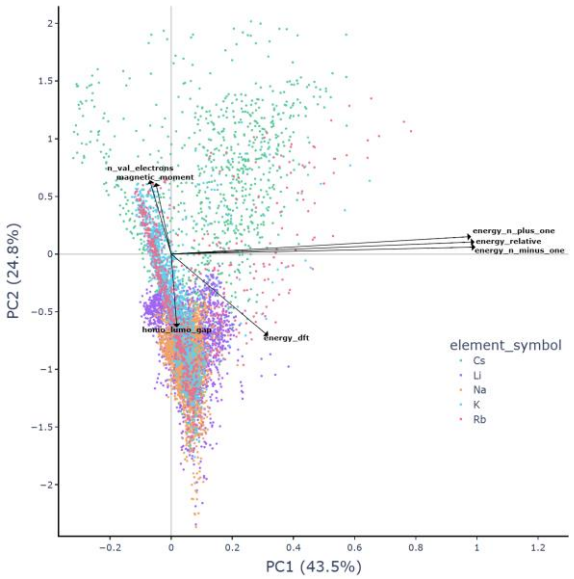
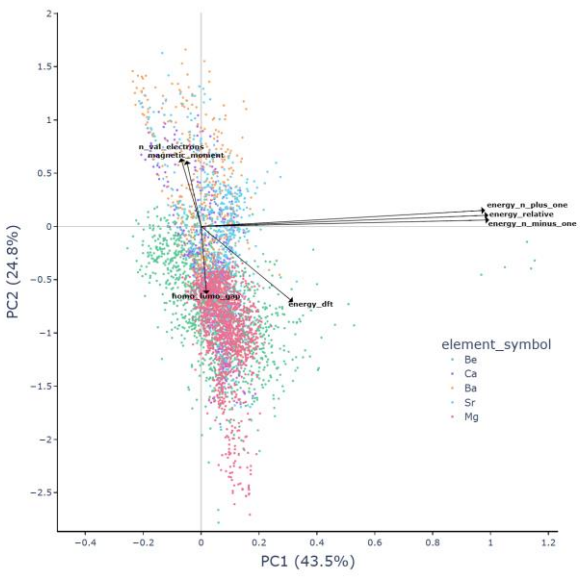


Fig.7 | PCA of properties in the Quantum Cluster Database. a–h PCA analysis of different families using PC1 and PC2 which explained 68.3% of the variance. **a** alkali metal, **b** alkaline earth **c** post transition metal, **d** metalloids, **e** nonmetal. Variables, or loadings components, are shown as black arrows of increasing value from the origin [0.0].

a



b



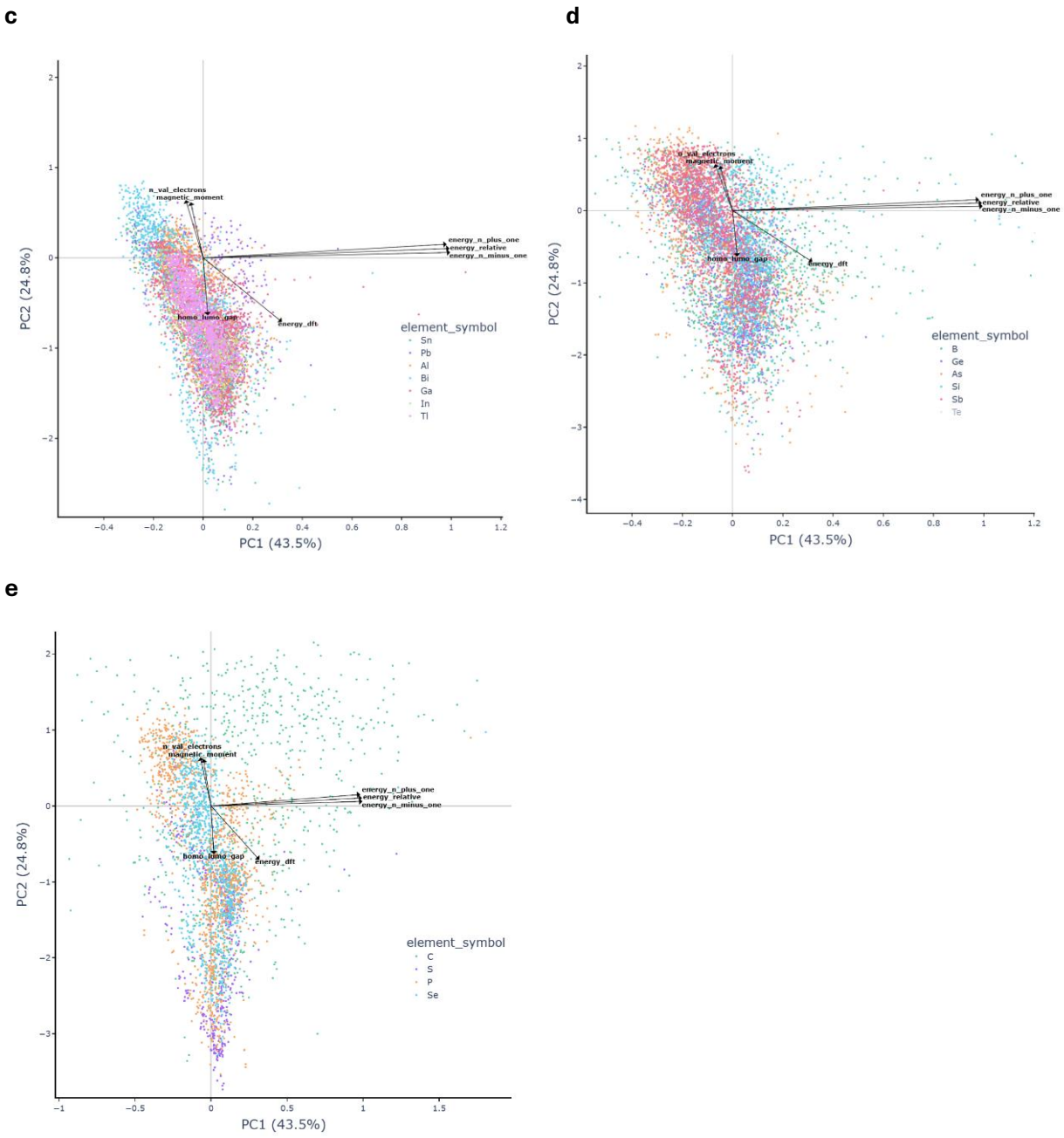


Fig.6 shows PC1 explained highest variance and with PC2, the total variance being explained is 68.3% which is considered acceptable but not ideal.

Fig.7 shows different element families have a different spread of density across the PCA plot. Looking at the principal axes, the first dimension mainly driven by energy n plus one, energy n minus one, and energy relative being the main drivers. The second dimension is driven by HOMO-LUMO gaps, magnetic moment, and number of valence electron. For example, looking at **Fig.1b**, all alkaline earth clusters are close in PC1, but spread out in PC2. Thus, although being in the same family, Mg have a higher HOMO-LUMO gap while Bg does not. Alkali metal and post transition metal have similar behavior, but with higher variety in PC1. Metalloids and nonmetals spread out much more evenly through PC1 and PC2, showing that these two groups possessed clusters that have a larger variety of properties and stability.

Fig.8 | PCA of all elements where the marker size increase with increasing number of atom.

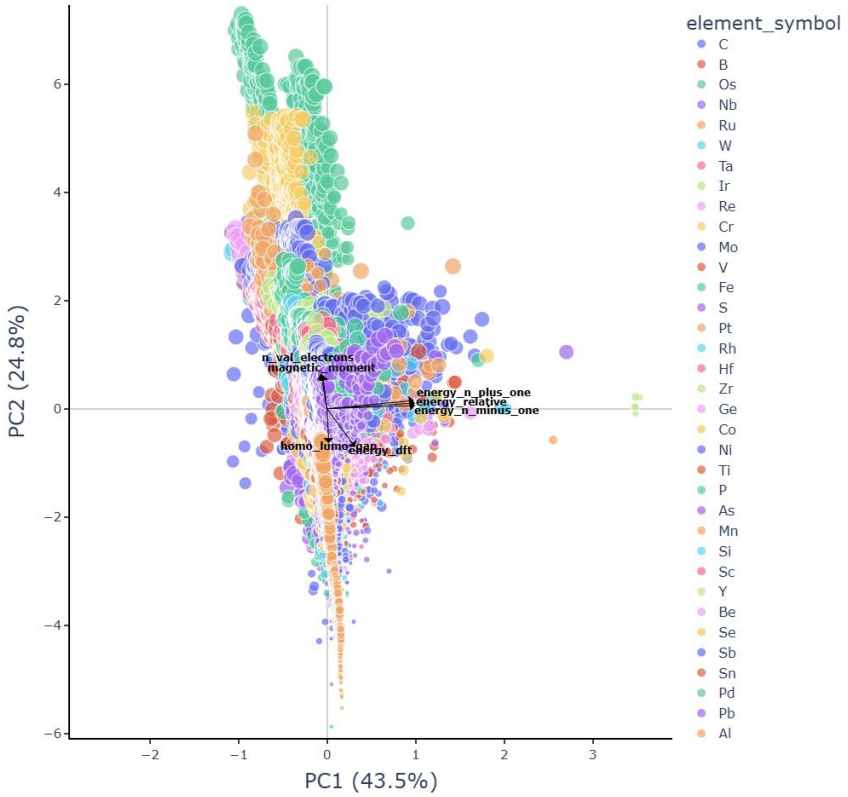
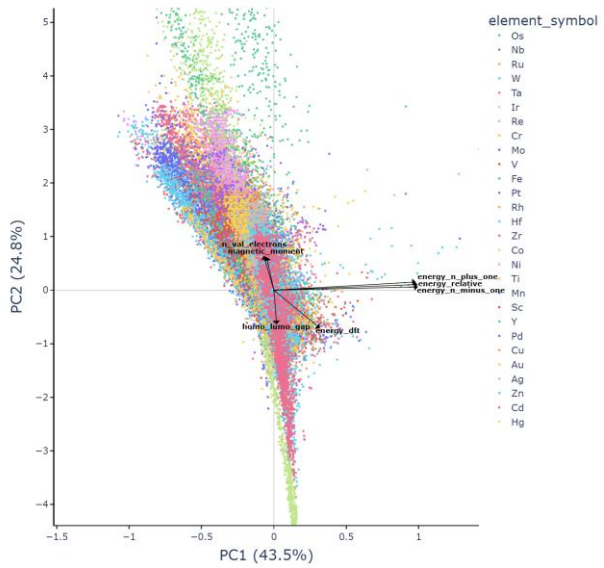


Fig.8 illustrates cluster's number of atoms increase with PC2, which is consistent with negative correlation found in **Fig.4**, and can be used as an indicator of stability when choosing cluster size within the same element.

Further Analysis of Transition Metals

Fig.9 | PCA of properties of transition metals. a zoom in view at the transition metal **b** transitions metal with number of atoms

a



b

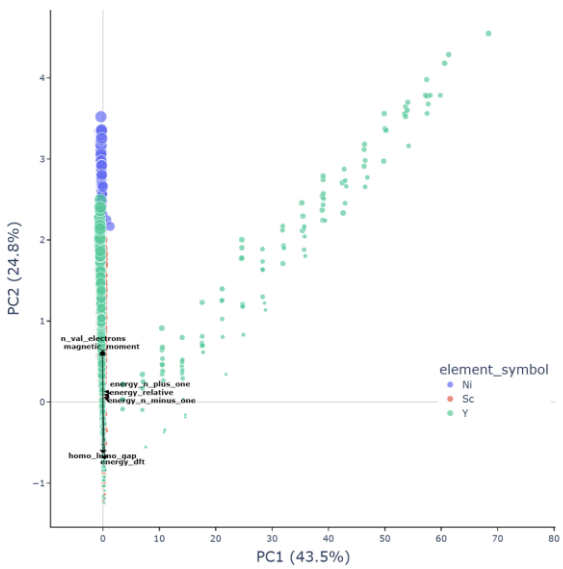


Fig.9 highlights distinct behavior in the transition metal group. When zoomed in **Fig. 9a**, this group behave simliar to the alkali metals but with a significant spread in PC2—much greater than the variation observed in all groups in **Fig.7**. However, when zoomed out, the transition metals appear unified along PC1, with the exception of a clear outlier, element Y, which shows a linear relationship between PC1 and PC2. **Fig.9b** shows that Y clusters with 10 to 20 atoms deviate from other Y clusters. Future investigations into the geometry of Y clusters specifically with 10 to 20 atoms is required to shed light on this deviation.

Conclusion

This analysis classifies nanoclusters into different families and shows most stable clusters fall within the first quadrant of the PCA plot in **Fig.7**, as they exhibit the lowest energy-DFT and the highest formation energy compared to other clusters. For example, if one is interested in studying quantum dots, focus on the PCA of metalloids (semiconducting materials) and select Si cluster in the first quadrant, which will give a stable Si nanocluster with 40-55 atoms, ideal for experimentation. For any experiment it is better to avoid using Y nanocluster due to its unpredictability.

Reference

1. Manna, Sukriti, et al. "A database of low-energy atomically precise nanoclusters." *Scientific Data* 10.1 (2023): 308.
2. Aihara, Jun-ichi. "Reduced HOMO− LUMO gap as an index of kinetic stability for polycyclic aromatic hydrocarbons." *The Journal of Physical Chemistry A* 103.37 (1999): 7487-7495.

Appendix

Python code for creating plots

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import colors
from matplotlib.ticker import PercentFormatter
import plotly.express as px
import seaborn as sb
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler #standardize the data so dataset have mean of zero
from sklearn.decomposition import PCA #Composition algorithm to perform PCA
import plotly.graph_objects as go
import plotly.io as pio

#Import Data
Nanocluster = pd.read_csv("Nanoclusterdata.csv")
df = Nanocluster
df.describe()

# Defined features and identities
features = ['energy_n_minus_one', 'energy_n_plus_one', 'homo_lumo_gap', 'n_val_electrons',
```

```
'magnetic_moment', 'energy_dft', 'energy_relative']
identities = ['n_atoms', 'cluster_id', 'element_symbol', 'structure_xyz', 'structure_poscar_format']

# Separating out the features
x = df.loc[:, features].values

# Separating out the identities
y = df.loc[:,identities].values

# Standardizing the features
x = StandardScaler().fit_transform(x)

# Perform PCA analysis on features
pca = PCA(n_components=6)

components = pca.fit_transform(x)

principalDf = pd.DataFrame(data = components
        , columns = ['PC1', 'PC2', 'PC3','PC4','PC5','PC6'])
# Add identities back to features to form final, transformed database of PCA analysis
finalDf = pd.concat([principalDf, df[identities]], axis = 1)

finalDf.head()

# Create Scree Plot
plt.figure(figsize=(10, 6))
per_var = np.round(pca.explained_variance_ratio_ * 100, decimals=1)
labels = ['PC' + str(x) for x in range(1, len(per_var) + 1)]

# Plot the bar chart
bars = plt.bar(x=range(1, len(per_var) + 1), height=per_var, tick_label=labels, color='skyblue')
plt.ylabel('Percentage of Variance (%)', fontsize=16)
plt.xlabel('Principal Component', fontsize=16)

# Adding percentage labels on top of each bar
for bar, percentage in zip(bars, per_var):
  plt.text(
    bar.get_x() + bar.get_width() / 2,   # X position (center of bar)
    bar.get_height(),                # Y position (top of the bar)
    f'{percentage}%',                 # Text (percentage)
    ha='center',                   # Horizontal alignment
    va='bottom',                    # Vertical alignment
    fontsize=12,                    # Font size
    color='black'                   # Text color
  )

# Customize the spines to remove only the x-axis spine
ax = plt.gca()  # Get current axis
```

```
ax.spines['top'].set_visible(False)    # Hide top spine
ax.spines['right'].set_visible(False)  # Hide right spine
ax.spines['left'].set_color('black')   # Set left spine color to black for visibility
ax.spines['bottom'].set_visible(False) # Hide bottom spine, keeping x-axis labels

# Save the plot
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\screeplot.png'
plt.savefig(file_path)

# Show plot
plt.show()

loadings = pca.components_.T * np.sqrt(pca.explained_variance_)

import matplotlib.pyplot as plt
import seaborn as sb
import pandas as pd

# Assuming df is already loaded

# Prepare sorted data for energy_dft based on unique element_symbol values
unique_elements = sorted(df['element_symbol'].unique())  # Sort unique element symbols

# Prepare data for each element
all_data = [df[df['element_symbol'] == el]['energy_dft'].values for el in unique_elements]

# Get the global y-axis limits (min and max values across all datasets)
y_min = min([min(data) for data in all_data])
y_max = max([max(data) for data in all_data])

# Set up the figure
fig, ax = plt.subplots(figsize=(18, 8), tight_layout=True)

# Plot the violin plot
parts = ax.violinplot(all_data, showmeans=False, showmedians=True, widths=0.8)

# Customize the violin plot appearance
for pc in parts['bodies']:
    pc.set_facecolor('#1f77b4')  # Consistent color for all violins
    pc.set_edgecolor('black')  # Black edge for definition
    pc.set_alpha(0.75)  # Transparency for a modern feel

# Customize the median lines
parts['cmedians'].set_edgecolor('black')
parts['cmedians'].set_linewidth(2)

# Set title and axis labels with increased font sizes
ax.set_xlabel('Element Symbol', fontsize=20)
```

```
ax.set_ylabel('DFT Energy', fontsize=20)
ax.set_xticks(range(1, len(unique_elements) + 1))
ax.set_xticklabels(unique_elements, rotation=45, ha='right', fontsize=16)

# Set the y-axis limits based on global values
ax.set_ylim(y_min, y_max)

# Add gridlines with larger font size for clarity
ax.yaxis.grid(True, linestyle='--', alpha=0.6)
ax.tick_params(axis='both', labelsize=15)  # Increase the size of ticks

# Remove unnecessary spines for a cleaner look
sb.despine(left=True, bottom=True)

# Save the plot
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Energy_Distribution_for_Elements.png'
plt.savefig(file_path)

# Show the plot
plt.tight_layout()  # Adjust layout for a clean, non-overlapping display
plt.show()

# Prepare sorted data for energy_dft based on unique n_atoms values
unique_n_atoms = sorted(df['n_atoms'].unique())  # Sort unique n_atoms values
all_data = [df[df['n_atoms'] == n]['energy_dft'].values for n in unique_n_atoms]

# Set up the figure and axis
fig, ax = plt.subplots(figsize=(12, 6))

# Plot the violin plot with refined aesthetics
parts = ax.violinplot(all_data, showmeans=False, showmedians=True, widths=0.8)

# Customize violin plot appearance for a professional look
for pc in parts['bodies']:
    pc.set_facecolor('#1f77b4')  # Use a single color for consistency
    pc.set_edgecolor('black')  # Black edge for definition
    pc.set_alpha(0.75)  # Transparency for a modern feel

# Customize median line
parts['cmedians'].set_edgecolor('black')
parts['cmedians'].set_linewidth(2)

# Set title and axis labels with professional font size
ax.set_title('Distribution of Energy (DFT) by Number of Atoms (n_atoms)')
ax.set_xlabel('Number of Atoms (n_atoms)', fontsize=14)
ax.set_ylabel('DFT Energy', fontsize=14)

# Set x-ticks to represent sorted unique values of `n_atoms`
```

```
ax.set_xticks(range(1, len(unique_n_atoms) + 1))
ax.set_xticklabels(unique_n_atoms, rotation=45, ha='right', fontsize=12)

# Add a light grid for clarity
ax.yaxis.grid(True, linestyle='--', alpha=0.6)

# Remove unnecessary spines for a cleaner look
sb.despine(left=True, bottom=True)

#save plot
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Distribution of Energy (DFT) by Number of Atoms (n_atoms)'
plt.savefig(file_path)

# Show the plot
plt.tight_layout()  # Adjust layout for a clean, non-overlapping display
plt.show()

# Prepare the selected columns
selected_columns = df[['energy_n_minus_one', 'energy_n_plus_one', 'homo_lumo_gap', 'n_val_electrons',
                       'magnetic_moment', 'energy_dft', 'energy_relative', 'n_atoms']]

# Prepare the selected columns
selected_columns = pd.concat([df.iloc[:, 1:6], df.iloc[:, 7:10]], axis=1)

# Create a pairplot with seaborn
sb.pairplot(selected_columns)

# Set the title and center it
plt.suptitle('Nanocluster Properties Scatter Matrix', fontsize=16, weight='bold', ha='center', va='top', y=1.05)

# Adjust the layout to avoid overlap
plt.tight_layout()

#save plot
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Nanocluster Properties Scatter Matrix'
plt.savefig(file_path)

# Show the plot
plt.show()

# Select columns 1 to 6 and 8 to 10
# selected_columns = pd.concat([df.iloc[:, 0:6], df.iloc[:, 7:10]], axis=1)

n_bins = 20  # Number of bins for the histogram
```

```
# Create distribution from your dataset (replace 'df['n_val_electrons']' with your column)
x = df['n_val_electrons']
legend = ['n_val_electrons distribution']

# Create histogram plot
fig, axs = plt.subplots(1, 1, figsize=(10, 7), tight_layout=True)

# Remove axes splines
for s in ['top', 'bottom', 'left', 'right']:
  axs.spines[s].set_visible(False)

# Remove x, y ticks
axs.xaxis.set_ticks_position('none')
axs.yaxis.set_ticks_position('none')

# Add padding between axes and labels
axs.xaxis.set_tick_params(pad=5)
axs.yaxis.set_tick_params(pad=10)

# Add x, y gridlines (corrected method)
axs.grid(True, color='grey', linestyle='-.', linewidth=0.5, alpha=0.6)

# Create histogram
N, bins, patches = axs.hist(x, bins=n_bins)

# Setting color
fracs = ((N**(1 / 5)) / N.max())  # Normalize the frequencies
norm = colors.Normalize(vmin=fracs.min(), vmax=fracs.max())  # Normalize for color mapping

# Apply color to each patch based on frequency
for thisfrac, thispatch in zip(fracs, patches):
  color = plt.cm.viridis(norm(thisfrac))  # Use colormap 'viridis'
  thispatch.set_facecolor(color)

# Adding extra features
axs.set_xlabel("Number of valence electrons", fontsize=24)  # X-axis label
axs.set_ylabel("Frequency", fontsize=24)  # Y-axis label
axs.tick_params(axis='x', labelsize=20)  # Increase x-axis tick size
axs.tick_params(axis='y', labelsize=20)  # Increase x-axis tick size

#save plot
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Frequency Distribution of number of
valence electrons'
plt.savefig(file_path)

# Show plot
plt.show()
```

```
# Select columns 1 to 6 and 8 to 10
# selected_columns = pd.concat([df.iloc[:, 0:6], df.iloc[:, 7:10]], axis=1)

n_bins = 20  # Number of bins for the histogram

# Create distribution from your dataset (replace 'df['n_val_electrons']' with your column)
x = df['homo_lumo_gap']

# Create histogram plot
fig, axs = plt.subplots(1, 1, figsize=(10, 7), tight_layout=True)

# Remove axes splines
for s in ['top', 'bottom', 'left', 'right']:
    axs.spines[s].set_visible(False)

# Remove x, y ticks
axs.xaxis.set_ticks_position('none')
axs.yaxis.set_ticks_position('none')

# Add padding between axes and labels
axs.xaxis.set_tick_params(pad=5)
axs.yaxis.set_tick_params(pad=10)

# Add x, y gridlines (corrected method)
axs.grid(True, color='grey', linestyle='-.', linewidth=0.5, alpha=0.6)

# Create histogram
N, bins, patches = axs.hist(x, bins=n_bins)

# Setting color
fracs = ((N**(1 / 5)) / N.max())  # Normalize the frequencies
norm = colors.Normalize(vmin=fracs.min(), vmax=fracs.max())  # Normalize for color mapping

# Apply color to each patch based on frequency
for thisfrac, thispatch in zip(fracs, patches):
    color = plt.cm.viridis(norm(thisfrac))  # Use colormap 'viridis'
    thispatch.set_facecolor(color)

# Adding extra features
axs.set_xlabel("HOMO-LUMO gap (eV)", fontsize=24)  # X-axis label
axs.set_ylabel("Frequency", fontsize=24)  # Y-axis label
axs.tick_params(axis='x', labelsize=20)  # Increase x-axis tick size
axs.tick_params(axis='y', labelsize=20)  # Increase x-axis tick size

# Define the path to the folder on your desktop
# Replace 'YourUsername' with your actual username and specify the folder path
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Frequency Distribution of HOMO-LUMO gap'
```

```
# Save the plot
plt.savefig(file_path)


# Show plot
plt.show()

n_bins = 6000  # Number of bins for the histogram

# Create distribution from your dataset (replace 'df['n_val_electrons']' with your column)
x = df['energy_n_minus_one']
legend = ['energy_n_minus_one distribution']

# Create histogram plot
fig, axs = plt.subplots(1, 1, figsize=(10, 7), tight_layout=True)

# Remove axes splines
for s in ['top', 'bottom', 'left', 'right']:
  axs.spines[s].set_visible(False)

# Remove x, y ticks
axs.xaxis.set_ticks_position('none')
axs.yaxis.set_ticks_position('none')

# Add padding between axes and labels
axs.xaxis.set_tick_params(pad=5)
axs.yaxis.set_tick_params(pad=10)

# Add x, y gridlines (corrected method)
axs.grid(True, color='grey', linestyle='-.', linewidth=0.5, alpha=0.6)

# Create histogram
N, bins, patches = axs.hist(x, bins=n_bins)

# Setting color
fracs = ((N**(1 / 5)) / N.max())  # Normalize the frequencies
norm = colors.Normalize(vmin=fracs.min(), vmax=fracs.max())  # Normalize for color mapping

# Apply color to each patch based on frequency
for thisfrac, thispatch in zip(fracs, patches):
  color = plt.cm.viridis(norm(thisfrac))  # Use colormap 'viridis'
  thispatch.set_facecolor(color)

# Adding extra features
axs.set_xlabel("Formation energy of N-1 (eV)", fontsize=24)  # X-axis label
axs.set_ylabel("Frequency", fontsize=24)  # Y-axis label)  # Y-axis label
axs.tick_params(axis='x', labelsize=20)  # Increase x-axis tick size
```

```
axs.tick_params(axis='y', labelsize=20)  # Increase x-axis tick size

# Show plot
plt.xlim(-10, 25)  # Sets the range of x-axis from 0 to 100

# Define the path to the folder on your desktop
# Replace 'YourUsername' with your actual username and specify the folder path
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Frequency Distribution of
energy_n_minus_one'

# Save the plot
plt.savefig(file_path)

plt.show()

n_bins = 6000  # Number of bins for the histogram

# Create distribution from your dataset (replace 'df['n_val_electrons']' with your column)
x = df['energy_n_plus_one']
legend = ['energy_n_minus_one distribution']

# Create histogram plot
fig, axs = plt.subplots(1, 1, figsize=(10, 7), tight_layout=True)

# Remove axes splines
for s in ['top', 'bottom', 'left', 'right']:
    axs.spines[s].set_visible(False)

# Remove x, y ticks
axs.xaxis.set_ticks_position('none')
axs.yaxis.set_ticks_position('none')

# Add padding between axes and labels
axs.xaxis.set_tick_params(pad=5)
axs.yaxis.set_tick_params(pad=10)

# Add x, y gridlines (corrected method)
axs.grid(True, color='grey', linestyle='-.', linewidth=0.5, alpha=0.6)

# Create histogram
N, bins, patches = axs.hist(x, bins=n_bins)

# Setting color
fracs = ((N**(1 / 5)) / N.max())  # Normalize the frequencies
norm = colors.Normalize(vmin=fracs.min(), vmax=fracs.max())  # Normalize for color mapping

# Apply color to each patch based on frequency
for thisfrac, thispatch in zip(fracs, patches):
```

```
    color = plt.cm.viridis(norm(thisfrac))  # Use colormap 'viridis'
    thispatch.set_facecolor(color)

# Adding extra features
axs.set_xlabel("Formation energy of N+1 (eV)", fontsize=24)  # X-axis label
axs.set_ylabel("Frequency", fontsize=24)  # Y-axis label)  # Y-axis label
axs.tick_params(axis='x', labelsize=20)  # Increase x-axis tick size
axs.tick_params(axis='y', labelsize=20)  # Increase x-axis tick size

# Show plot
plt.xlim(-10, 25)  # Sets the range of x-axis from 0 to 100

# Define the path to the folder on your desktop
# Replace 'YourUsername' with your actual username and specify the folder path
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Frequency Distribution of
energy_n_plus_one'

# Save the plot
plt.savefig(file_path)

plt.show()

# Select columns 1 to 6 and 8 to 10
# selected_columns = pd.concat([df.iloc[:, 0:6], df.iloc[:, 7:10]], axis=1)

n_bins = 20  # Number of bins for the histogram

# Create distribution from your dataset (replace 'df['n_val_electrons']' with your column)
x = df['magnetic_moment']
legend = ['n_val_electrons distribution']

# Create histogram plot
fig, axs = plt.subplots(1, 1, figsize=(10, 7), tight_layout=True)

# Remove axes splines
for s in ['top', 'bottom', 'left', 'right']:
    axs.spines[s].set_visible(False)

# Remove x, y ticks
axs.xaxis.set_ticks_position('none')
axs.yaxis.set_ticks_position('none')

# Add padding between axes and labels
axs.xaxis.set_tick_params(pad=5)
axs.yaxis.set_tick_params(pad=10)

# Add x, y gridlines (corrected method)
axs.grid(True, color='grey', linestyle='-.', linewidth=0.5, alpha=0.6)
```

```
# Create histogram
N, bins, patches = axs.hist(x, bins=n_bins)

# Setting color
fracs = ((N**(1 / 5)) / N.max())  # Normalize the frequencies
norm = colors.Normalize(vmin=fracs.min(), vmax=fracs.max())  # Normalize for color mapping

# Apply color to each patch based on frequency
for thisfrac, thispatch in zip(fracs, patches):
    color = plt.cm.viridis(norm(thisfrac))  # Use colormap 'viridis'
    thispatch.set_facecolor(color)

# Adding extra features
axs.set_xlabel("Magnetic moment (μB)", fontsize=24)  # X-axis label
axs.set_ylabel("Frequency", fontsize=24)  # Y-axis label)  # Y-axis label
axs.tick_params(axis='x', labelsize=20)  # Increase x-axis tick size
axs.tick_params(axis='y', labelsize=20)  # Increase x-axis tick size


# Define the path to the folder on your desktop
# Replace 'YourUsername' with your actual username and specify the folder path
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Frequency Distribution of magnetic
moment'

# Save the plot
plt.savefig(file_path)


# Show plot
plt.show()

# Select columns 1 to 6 and 8 to 10
# selected_columns = pd.concat([df.iloc[:, 0:6], df.iloc[:, 7:10]], axis=1)

n_bins = 55  # Number of bins for the histogram

# Create distribution from your dataset (replace 'df['n_val_electrons']' with your column)
x = df['n_atoms']
legend = ['n_val_electrons distribution']

# Create histogram plot
fig, axs = plt.subplots(1, 1, figsize=(10, 7), tight_layout=True)

# Remove axes splines
for s in ['top', 'bottom', 'left', 'right']:
    axs.spines[s].set_visible(False)
```

```
# Remove x, y ticks
axs.xaxis.set_ticks_position('none')
axs.yaxis.set_ticks_position('none')

# Add padding between axes and labels
axs.xaxis.set_tick_params(pad=5)
axs.yaxis.set_tick_params(pad=10)

# Add x, y gridlines (corrected method)
axs.grid(True, color='grey', linestyle='-.', linewidth=0.5, alpha=0.6)

# Create histogram
N, bins, patches = axs.hist(x, bins=n_bins)

# Setting color
fracs = ((N**(1 / 5)) / N.max())  # Normalize the frequencies
norm = colors.Normalize(vmin=fracs.min(), vmax=fracs.max())  # Normalize for color mapping

# Apply color to each patch based on frequency
for thisfrac, thispatch in zip(fracs, patches):
    color = plt.cm.viridis(norm(thisfrac))  # Use colormap 'viridis'
    thispatch.set_facecolor(color)

# Adding extra features
axs.set_xlabel("Number of atoms", fontsize=24)  # X-axis label
axs.set_ylabel("Frequency", fontsize=24)  # Y-axis label)  # Y-axis label
axs.tick_params(axis='x', labelsize=20)  # Increase x-axis tick size
axs.tick_params(axis='y', labelsize=20)  # Increase x-axis tick size

# Define the path to the folder on your desktop
# Replace 'YourUsername' with your actual username and specify the folder path
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Frequency Distribution of Number of
atoms'

# Save the plot
plt.savefig(file_path)


# Show plot
plt.show()

# Select columns 1 to 6 and 8 to 10
# selected_columns = pd.concat([df.iloc[:, 0:6], df.iloc[:, 7:10]], axis=1)

n_bins = 100  # Number of bins for the histogram

# Create distribution from your dataset (replace 'df['n_val_electrons']' with your column)
x = df['energy_dft']
```

```
# Create histogram plot
fig, axs = plt.subplots(1, 1, figsize=(10, 7), tight_layout=True)

# Remove axes splines
for s in ['top', 'bottom', 'left', 'right']:
    axs.spines[s].set_visible(False)

# Remove x, y ticks
axs.xaxis.set_ticks_position('none')
axs.yaxis.set_ticks_position('none')

# Add padding between axes and labels
axs.xaxis.set_tick_params(pad=5)
axs.yaxis.set_tick_params(pad=10)

# Add x, y gridlines (corrected method)
axs.grid(True, color='grey', linestyle='-.', linewidth=0.5, alpha=0.6)

# Create histogram
N, bins, patches = axs.hist(x, bins=n_bins)

# Setting color
fracs = ((N**(1 / 5)) / N.max())  # Normalize the frequencies
norm = colors.Normalize(vmin=fracs.min(), vmax=fracs.max())  # Normalize for color mapping

# Apply color to each patch based on frequency
for thisfrac, thispatch in zip(fracs, patches):
    color = plt.cm.viridis(norm(thisfrac))  # Use colormap 'viridis'
    thispatch.set_facecolor(color)

# Adding extra features
axs.set_xlabel("Energy calculated using DFT (eV)", fontsize=24)  # X-axis label
axs.set_ylabel("Frequency", fontsize=24)  # Y-axis label
axs.tick_params(axis='x', labelsize=20)  # Increase x-axis tick size
axs.tick_params(axis='y', labelsize=20)  # Increase x-axis tick size


# Define the path to the folder on your desktop
# Replace 'YourUsername' with your actual username and specify the folder path
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Frequency Distribution of energy
calculated using Density Functional Theory (DFT)'

# Save the plot
plt.savefig(file_path)


# Show plot
```

```
plt.show()

# Select columns 1 to 6 and 8 to 10
# selected_columns = pd.concat([df.iloc[:, 0:6], df.iloc[:, 7:10]], axis=1)

n_bins = 5000  # Number of bins for the histogram

# Create distribution from your dataset (replace 'df['n_val_electrons']' with your column)
x = df['energy_relative']

# Create histogram plot
fig, axs = plt.subplots(1, 1, figsize=(10, 7), tight_layout=True)

# Remove axes splines
for s in ['top', 'bottom', 'left', 'right']:
  axs.spines[s].set_visible(False)

# Remove x, y ticks
axs.xaxis.set_ticks_position('none')
axs.yaxis.set_ticks_position('none')

# Add padding between axes and labels
axs.xaxis.set_tick_params(pad=5)
axs.yaxis.set_tick_params(pad=10)

# Add x, y gridlines (corrected method)
axs.grid(True, color='grey', linestyle='-.', linewidth=0.5, alpha=0.6)

# Create histogram
N, bins, patches = axs.hist(x, bins=n_bins)

# Setting color
fracs = ((N**(1 / 5)) / N.max())  # Normalize the frequencies
norm = colors.Normalize(vmin=fracs.min(), vmax=fracs.max())  # Normalize for color mapping

# Apply color to each patch based on frequency
for thisfrac, thispatch in zip(fracs, patches):
  color = plt.cm.viridis(norm(thisfrac))  # Use colormap 'viridis'
  thispatch.set_facecolor(color)

# Adding extra features
axs.set_xlabel("Relative energy (eV)", fontsize=24)  # X-axis label
axs.set_ylabel("Frequency", fontsize=24)  # Y-axis label
axs.tick_params(axis='x', labelsize=20)  # Increase x-axis tick size
axs.tick_params(axis='y', labelsize=20)  # Increase x-axis tick size
plt.xlim(-10, 20)  # Sets the range of x-axis from 0 to 100
```

```
# Define the path to the folder on your desktop
# Replace 'YourUsername' with your actual username and specify the folder path
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Frequency Distribution of Energy above the lowest energy structure of the same element and size (eV)'

# Save the plot
plt.savefig(file_path)


# Show plot
plt.show()

# Count frequency of each element symbol
element_counts = df['element_symbol'].value_counts().sort_index()

# Define the number of unique elements
n_elements = element_counts.shape[0]

# Create bar plot with the same style elements
fig, axs = plt.subplots(1, 1, figsize=(15, 7), tight_layout=True)

# Remove axes splines
for s in ['top', 'bottom', 'left', 'right']:
    axs.spines[s].set_visible(False)

# Remove x, y ticks
axs.xaxis.set_ticks_position('none')
axs.yaxis.set_ticks_position('none')

# Add padding between axes and labels
axs.xaxis.set_tick_params(pad=5)
axs.yaxis.set_tick_params(pad=10)

# Add x, y gridlines
axs.grid(True, color='grey', linestyle='-.', linewidth=0.5, alpha=0.6)

# Set colors for each bar based on normalized frequencies
fracs = (element_counts.values ** (1 / 5)) / element_counts.values.max()
norm = colors.Normalize(vmin=fracs.min(), vmax=fracs.max())

# Plot each bar with its calculated color from the colormap
bars = axs.bar(element_counts.index, element_counts.values, color=[plt.cm.viridis(norm(f)) for f in fracs])

# Rotate x-axis labels for better readability
plt.xticks(rotation=90)

# Adding labels and title
```

```
axs.set_xlabel("Element Symbol", fontsize=24)   # X-axis label
axs.set_ylabel("Frequency", fontsize=24)       # Y-axis label
axs.tick_params(axis='x', labelsize=20)  # Increase x-axis tick size
axs.tick_params(axis='y', labelsize=20)  # Increase x-axis tick size

# Save the plot to the specified path on your desktop
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Frequency Distribution of
element_symbol'
plt.savefig(file_path)

# Show plot
plt.show()

# Set the size of the figure
plt.figure(figsize=(12, 8))

# Print data that will be plotted
print(selected_columns.corr(numeric_only=True))

# Plotting correlation heatmap with customized aesthetics
dataplot = sb.heatmap(
   selected_columns.corr(numeric_only=True),
   cmap="coolwarm",          # 'coolwarm' colormap for contrast
   annot=True,              # Show correlation coefficients on heatmap
   fmt=".2f",               # Format annotations to 2 decimal places
   annot_kws={"size": 10, "color": "black"},  # Customize annotation style
   linewidths=0.5,          # Line width between cells
   linecolor='black',       # Line color
   cbar_kws={"shrink": .8, "label": "Correlation Coefficient"},  # Color bar label
)

# Set title with matching font size and style
plt.title("Correlation Heatmap of Selected Features", fontsize=16)

# Adjust x and y ticks for consistency
plt.xticks(rotation=45, ha='right', fontsize=12)
plt.yticks(rotation=0, fontsize=12)

# Define the path to the folder on your desktop
file_path = r'C:\Users\Eileen\Desktop\CENG0068_CW1_MBKX3\Correlation Heatmap of Selected
Features'

# Save the plot
plt.savefig(file_path)

# Display heatmap with tight layout
plt.tight_layout()  # Adjust layout to fit everything nicely
plt.show()
```

```
# Define a list of element symbols in the same group to include in the plot
alkali_Metals = ["Li","Na","K","Rb","Cs","Fr"]
alkaline_Earths = ["Be","Mg","Ca","Sr","Ba","Ra"]
transition_metals = [
  "Sc", "Ti", "V", "Cr", "Mn", "Fe", "Co", "Ni", "Cu", "Zn",   # Period 4
  "Y", "Zr", "Nb", "Mo", "Tc", "Ru", "Rh", "Pd", "Ag", "Cd",   # Period 5
  "Hf", "Ta", "W", "Re", "Os", "Ir", "Pt", "Au", "Hg",       # Period 6
  "Rf", "Db", "Sg", "Bh", "Hs", "Mt", "Ds", "Rg", "Cn"       # Period 7
]
post_transition_metals = [
  "Al", "Ga", "In", "Tl",  # Group 13
  "Sn", "Pb", "Fl",       # Group 14
  "Bi", "Po", "Lv"        # Group 15
]
metalloids = [
  "B", "Si", "Ge", "As", "Sb", "Te", "Po", "At"
]
nonmetals = [
  "H", "C", "N", "O", "F", "P", "S", "Cl", "Se", "Br", "I", "He", "Ne", "Ar", "Kr", "Xe", "Rn"
]

# Filter the dataframe for rows with the specified element symbols
filtered_df = finalDf[finalDf["element_symbol"].isin(alkali_Metals)]

# Create the scatter plot
fig = px.scatter(
  filtered_df,
  x="PC1",
  y="PC2",
  width=1000,
  height=1000,
  color="element_symbol",
)


# Set marker size to constant
fig.update_traces(marker=dict(size=4))

#Update axis title and font size
fig.update_layout(
  xaxis_title="PC1 (43.5%)",  # Custom label for x-axis
  yaxis_title="PC2 (24.8%)",  # Custom label for y-axis
  xaxis=dict(
    title_font=dict(size=24),  # Increase font size of x-axis title
    tickfont=dict(size=15),    # Increase font size of x-axis tick labels
  ),
  yaxis=dict(
```

```
      title_font=dict(size=24),  # Increase font size of y-axis title
      tickfont=dict(size=15),    # Increase font size of y-axis tick labels
    ),
    legend=dict(
      title=dict(font=dict(size=22)),  # Font size for the legend title
      font=dict(size=16),           # Font size for the legend items
)
)

# Update the axis lines (spines)
fig.update_layout(
    xaxis=dict(
      showline=True,         # Show bottom line (x-axis)
      linewidth=2,           # Set line width for the bottom line
      linecolor="black",      # Set line color for the bottom line
      ticks="outside",       # Place ticks outside of the axis
      tickwidth=2,           # Set tick width
      showgrid=False,        # Hide gridlines on the x-axis
    ),
    yaxis=dict(
      showline=True,         # Show left line (y-axis)
      linewidth=2,           # Set line width for the left line
      linecolor="black",      # Set line color for the left line
      ticks="outside",       # Place ticks outside of the axis
      tickwidth=2,           # Set tick width
      showgrid=False,        # Hide gridlines on the y-axis
    ),
    plot_bgcolor='white'        # Set the background color to white
)

# Add zero lines
fig.update_xaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')
fig.update_yaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')

# Adjust annotation positioning to avoid overlap
for i, feature in enumerate(features):
    # Set individual shifts for each feature
    if feature == "energy_n_minus_one":
      xshift, yshift = 72, -15 # Example shifts for feature1
    elif feature == "energy_relative":
      xshift, yshift = 55, -8  # Example shifts for feature2
    elif feature == "energy_n_plus_one":
      xshift, yshift = 70, 0  # Example shifts for feature3
    elif feature == "n_val_electrons":
      xshift, yshift = -15, 10  # Example shifts for feature3
    elif feature == "magnetic_moment":
      xshift, yshift = -1, 0 # Example shifts for feature3
    elif feature == "homo_lumo_gap":
```

```
        xshift, yshift = 0, -13  # Example shifts for feature3
    else:
        xshift, yshift = 30, -13    # Default shifts for others

    # Add arrows for the loadings
    fig.add_annotation(
        ax=0, ay=0,
        axref="x", ayref="y",
        x=loadings[i, 0],
        y=loadings[i, 1],
        showarrow=True,
        arrowsize=2,
        arrowhead=1,
        xanchor="left",
        yanchor="bottom",
        arrowcolor="black"  # Change to desired color for the arrow
    )

    # Add text annotations with individual shifts
    fig.add_annotation(
        x=loadings[i, 0],
        y=loadings[i, 1],
        ax=0, ay=0,
        xanchor="center",  # Adjust horizontal anchor
        yanchor="bottom",   # Adjust vertical anchor
        text=f"<b>{feature}</b>",
        yshift=yshift,    # Individual y shift
        xshift=xshift ,    # Individual x shift
        font=dict(
        color="black",    # Set text color
    ),

    )
fig.update_layout(legend=dict(
    yanchor="bottom",
    y=0.20,
    xanchor="right",
    x=0.99
))

fig.show()

# Filter the dataframe for rows with the specified element symbols
filtered_df = finalDf[finalDf["element_symbol"].isin(alkaline_Earths)]



# Create the scatter plot
fig = px.scatter(
```

```
    filtered_df,
    x="PC1",
    y="PC2",
    width=1000,
    height=1000,
    color="element_symbol",
)


# Set marker size to constant
fig.update_traces(marker=dict(size=4))

#Update axis title and font size
fig.update_layout(
    xaxis_title="PC1 (43.5%)",  # Custom label for x-axis
    yaxis_title="PC2 (24.8%)",  # Custom label for y-axis
    xaxis=dict(
        title_font=dict(size=24),  # Increase font size of x-axis title
        tickfont=dict(size=15),    # Increase font size of x-axis tick labels
    ),
    yaxis=dict(
        title_font=dict(size=24),  # Increase font size of y-axis title
        tickfont=dict(size=15),    # Increase font size of y-axis tick labels
    ),
    legend=dict(
        title=dict(font=dict(size=22)),  # Font size for the legend title
        font=dict(size=16),           # Font size for the legend items
)
)

# Update the axis lines (spines)
fig.update_layout(
    xaxis=dict(
        showline=True,         # Show bottom line (x-axis)
        linewidth=2,           # Set line width for the bottom line
        linecolor="black",      # Set line color for the bottom line
        ticks="outside",       # Place ticks outside of the axis
        tickwidth=2,           # Set tick width
        showgrid=False,        # Hide gridlines on the x-axis
    ),
    yaxis=dict(
        showline=True,         # Show left line (y-axis)
        linewidth=2,           # Set line width for the left line
        linecolor="black",      # Set line color for the left line
        ticks="outside",       # Place ticks outside of the axis
        tickwidth=2,           # Set tick width
        showgrid=False,        # Hide gridlines on the y-axis
    ),
```

```
    plot_bgcolor='white'         # Set the background color to white
)

# Add zero lines
fig.update_xaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')
fig.update_yaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')

# Adjust annotation positioning to avoid overlap
for i, feature in enumerate(features):
    # Set individual shifts for each feature
    if feature == "energy_n_minus_one":
        xshift, yshift = 72, -15 # Example shifts for feature1
    elif feature == "energy_relative":
        xshift, yshift = 55, -8  # Example shifts for feature2
    elif feature == "energy_n_plus_one":
        xshift, yshift = 70, 0  # Example shifts for feature3
    elif feature == "n_val_electrons":
        xshift, yshift = -15, 10  # Example shifts for feature3
    elif feature == "magnetic_moment":
        xshift, yshift = -1, 0 # Example shifts for feature3
    elif feature == "homo_lumo_gap":
        xshift, yshift = 0, -13  # Example shifts for feature3
    else:
        xshift, yshift = 30, -13    # Default shifts for others

    # Add arrows for the loadings
    fig.add_annotation(
        ax=0, ay=0,
        axref="x", ayref="y",
        x=loadings[i, 0],
        y=loadings[i, 1],
        showarrow=True,
        arrowsize=2,
        arrowhead=1,
        xanchor="left",
        yanchor="bottom",
        arrowcolor="black"  # Change to desired color for the arrow
    )

    # Add text annotations with individual shifts
    fig.add_annotation(
        x=loadings[i, 0],
        y=loadings[i, 1],
        ax=0, ay=0,
        xanchor="center",  # Adjust horizontal anchor
        yanchor="bottom",   # Adjust vertical anchor
        text=f"<b>{feature}</b>",
        yshift=yshift,     # Individual y shift
```

```
      xshift=xshift ,    # Individual x shift
      font=dict(
      color="black",   # Set text color
    ),

    )

fig.update_layout(legend=dict(
    yanchor="bottom",
    y=0.20,
    xanchor="right",
    x=0.99
))

fig.show()

# Filter the dataframe for rows with the specified element symbols
filtered_df = finalDf[finalDf["element_symbol"].isin(transition_metals)]


# Create the scatter plot
fig = px.scatter(
    filtered_df,
    x="PC1",
    y="PC2",
    width=1000,
    height=1000,
    color="element_symbol",
)


# Set marker size to constant
fig.update_traces(marker=dict(size=4))

#Update axis title and font size
fig.update_layout(
    xaxis_title="PC1 (43.5%)",  # Custom label for x-axis
    yaxis_title="PC2 (24.8%)",  # Custom label for y-axis
    xaxis=dict(
        title_font=dict(size=24),  # Increase font size of x-axis title
        tickfont=dict(size=15),    # Increase font size of x-axis tick labels
    ),
    yaxis=dict(
        title_font=dict(size=24),  # Increase font size of y-axis title
        tickfont=dict(size=15),    # Increase font size of y-axis tick labels
    ),
    legend=dict(
        title=dict(font=dict(size=22)),  # Font size for the legend title
```

```
    font=dict(size=16),          # Font size for the legend items
)
)

# Update the axis lines (spines)
fig.update_layout(
  xaxis=dict(
    showline=True,         # Show bottom line (x-axis)
    linewidth=2,           # Set line width for the bottom line
    linecolor="black",     # Set line color for the bottom line
    ticks="outside",       # Place ticks outside of the axis
    tickwidth=2,           # Set tick width
    showgrid=False,        # Hide gridlines on the x-axis
  ),
  yaxis=dict(
    showline=True,         # Show left line (y-axis)
    linewidth=2,           # Set line width for the left line
    linecolor="black",     # Set line color for the left line
    ticks="outside",       # Place ticks outside of the axis
    tickwidth=2,           # Set tick width
    showgrid=False,        # Hide gridlines on the y-axis
  ),
  plot_bgcolor='white'       # Set the background color to white
)

# Add zero lines
fig.update_xaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')
fig.update_yaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')

# Adjust annotation positioning to avoid overlap
for i, feature in enumerate(features):
  # Set individual shifts for each feature
  if feature == "energy_n_minus_one":
    xshift, yshift = 72, -15 # Example shifts for feature1
  elif feature == "energy_relative":
    xshift, yshift = 55, -8  # Example shifts for feature2
  elif feature == "energy_n_plus_one":
    xshift, yshift = 70, 0  # Example shifts for feature3
  elif feature == "n_val_electrons":
    xshift, yshift = -15, 10  # Example shifts for feature3
  elif feature == "magnetic_moment":
    xshift, yshift = -1, 0 # Example shifts for feature3
  elif feature == "homo_lumo_gap":
    xshift, yshift = 0, -13  # Example shifts for feature3
  else:
    xshift, yshift = 30, -13   # Default shifts for others

  # Add arrows for the loadings
```

```
    fig.add_annotation(
        ax=0, ay=0,
        axref="x", ayref="y",
        x=loadings[i, 0],
        y=loadings[i, 1],
        showarrow=True,
        arrowsize=2,
        arrowhead=1,
        xanchor="left",
        yanchor="bottom",
        arrowcolor="black"  # Change to desired color for the arrow
    )

    # Add text annotations with individual shifts
    fig.add_annotation(
        x=loadings[i, 0],
        y=loadings[i, 1],
        ax=0, ay=0,
        xanchor="center",  # Adjust horizontal anchor
        yanchor="bottom",   # Adjust vertical anchor
        text=f"<b>{feature}</b>",
        yshift=yshift,    # Individual y shift
        xshift=xshift ,    # Individual x shift
        font=dict(
        color="black",    # Set text color
    ),

    )


fig.show()

# Filter the dataframe for rows with the specified element symbols
filtered_df = finalDf[finalDf["element_symbol"].isin(post_transition_metals)]

# Create the scatter plot
fig = px.scatter(
    filtered_df,
    x="PC1",
    y="PC2",
    width=1000,
    height=1000,
    color="element_symbol",
)


# Set marker size to constant
fig.update_traces(marker=dict(size=4))
```

```
#Update axis title and font size
fig.update_layout(
  xaxis_title="PC1 (43.5%)",  # Custom label for x-axis
  yaxis_title="PC2 (24.8%)",  # Custom label for y-axis
  xaxis=dict(
    title_font=dict(size=24),  # Increase font size of x-axis title
    tickfont=dict(size=15),    # Increase font size of x-axis tick labels
  ),
  yaxis=dict(
    title_font=dict(size=24),  # Increase font size of y-axis title
    tickfont=dict(size=15),    # Increase font size of y-axis tick labels
  ),
  legend=dict(
    title=dict(font=dict(size=22)),  # Font size for the legend title
    font=dict(size=16),           # Font size for the legend items
)
)

# Update the axis lines (spines)
fig.update_layout(
  xaxis=dict(
    showline=True,         # Show bottom line (x-axis)
    linewidth=2,           # Set line width for the bottom line
    linecolor="black",      # Set line color for the bottom line
    ticks="outside",       # Place ticks outside of the axis
    tickwidth=2,           # Set tick width
    showgrid=False,        # Hide gridlines on the x-axis
  ),
  yaxis=dict(
    showline=True,         # Show left line (y-axis)
    linewidth=2,           # Set line width for the left line
    linecolor="black",      # Set line color for the left line
    ticks="outside",       # Place ticks outside of the axis
    tickwidth=2,           # Set tick width
    showgrid=False,        # Hide gridlines on the y-axis
  ),
  plot_bgcolor='white'        # Set the background color to white
)

# Add zero lines
fig.update_xaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')
fig.update_yaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')

# Adjust annotation positioning to avoid overlap
for i, feature in enumerate(features):
  # Set individual shifts for each feature
  if feature == "energy_n_minus_one":
```

```
        xshift, yshift = 72, -15 # Example shifts for feature1
    elif feature == "energy_relative":
        xshift, yshift = 55, -8  # Example shifts for feature2
    elif feature == "energy_n_plus_one":
        xshift, yshift = 70, 0  # Example shifts for feature3
    elif feature == "n_val_electrons":
        xshift, yshift = -15, 10  # Example shifts for feature3
    elif feature == "magnetic_moment":
        xshift, yshift = -1, 0 # Example shifts for feature3
    elif feature == "homo_lumo_gap":
        xshift, yshift = 0, -13  # Example shifts for feature3
    else:
        xshift, yshift = 30, -13    # Default shifts for others

    # Add arrows for the loadings
    fig.add_annotation(
        ax=0, ay=0,
        axref="x", ayref="y",
        x=loadings[i, 0],
        y=loadings[i, 1],
        showarrow=True,
        arrowsize=2,
        arrowhead=1,
        xanchor="left",
        yanchor="bottom",
        arrowcolor="black"  # Change to desired color for the arrow
    )

    # Add text annotations with individual shifts
    fig.add_annotation(
        x=loadings[i, 0],
        y=loadings[i, 1],
        ax=0, ay=0,
        xanchor="center",  # Adjust horizontal anchor
        yanchor="bottom",   # Adjust vertical anchor
        text=f"<b>{feature}</b>",
        yshift=yshift,    # Individual y shift
        xshift=xshift ,    # Individual x shift
        font=dict(
        color="black",    # Set text color
    ),

    )
fig.update_layout(legend=dict(
    yanchor="bottom",
    y=0.20,
    xanchor="right",
    x=0.99
```

```
))
fig.show()

# Filter the dataframe for rows with the specified element symbols
filtered_df = finalDf[finalDf["element_symbol"].isin(metalloids)]

# Create the scatter plot
fig = px.scatter(
  filtered_df,
  x="PC1",
  y="PC2",
  width=1000,
  height=1000,
  color="element_symbol",
)


# Set marker size to constant
fig.update_traces(marker=dict(size=4))

#Update axis title and font size
fig.update_layout(
  xaxis_title="PC1 (43.5%)",  # Custom label for x-axis
  yaxis_title="PC2 (24.8%)",  # Custom label for y-axis
  xaxis=dict(
    title_font=dict(size=24),  # Increase font size of x-axis title
    tickfont=dict(size=15),    # Increase font size of x-axis tick labels
  ),
  yaxis=dict(
    title_font=dict(size=24),  # Increase font size of y-axis title
    tickfont=dict(size=15),    # Increase font size of y-axis tick labels
  ),
  legend=dict(
    title=dict(font=dict(size=22)),  # Font size for the legend title
    font=dict(size=16),           # Font size for the legend items
)
)

# Update the axis lines (spines)
fig.update_layout(
  xaxis=dict(
    showline=True,         # Show bottom line (x-axis)
    linewidth=2,           # Set line width for the bottom line
    linecolor="black",      # Set line color for the bottom line
    ticks="outside",       # Place ticks outside of the axis
    tickwidth=2,           # Set tick width
    showgrid=False,        # Hide gridlines on the x-axis
  ),
```

```
    yaxis=dict(
        showline=True,          # Show left line (y-axis)
        linewidth=2,            # Set line width for the left line
        linecolor="black",      # Set line color for the left line
        ticks="outside",        # Place ticks outside of the axis
        tickwidth=2,            # Set tick width
        showgrid=False,         # Hide gridlines on the y-axis
    ),
    plot_bgcolor='white'        # Set the background color to white
)

# Add zero lines
fig.update_xaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')
fig.update_yaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')

# Adjust annotation positioning to avoid overlap
for i, feature in enumerate(features):
    # Set individual shifts for each feature
    if feature == "energy_n_minus_one":
        xshift, yshift = 72, -15 # Example shifts for feature1
    elif feature == "energy_relative":
        xshift, yshift = 55, -8  # Example shifts for feature2
    elif feature == "energy_n_plus_one":
        xshift, yshift = 70, 0  # Example shifts for feature3
    elif feature == "n_val_electrons":
        xshift, yshift = -15, 10  # Example shifts for feature3
    elif feature == "magnetic_moment":
        xshift, yshift = -1, 0 # Example shifts for feature3
    elif feature == "homo_lumo_gap":
        xshift, yshift = 0, -13  # Example shifts for feature3
    else:
        xshift, yshift = 30, -13    # Default shifts for others

    # Add arrows for the loadings
    fig.add_annotation(
        ax=0, ay=0,
        axref="x", ayref="y",
        x=loadings[i, 0],
        y=loadings[i, 1],
        showarrow=True,
        arrowsize=2,
        arrowhead=1,
        xanchor="left",
        yanchor="bottom",
        arrowcolor="black"  # Change to desired color for the arrow
    )

    # Add text annotations with individual shifts
```

```
    fig.add_annotation(
        x=loadings[i, 0],
        y=loadings[i, 1],
        ax=0, ay=0,
        xanchor="center",  # Adjust horizontal anchor
        yanchor="bottom",   # Adjust vertical anchor
        text=f"<b>{feature}</b>",
        yshift=yshift,    # Individual y shift
        xshift=xshift ,    # Individual x shift
        font=dict(
        color="black",    # Set text color
    ),

    )

fig.update_layout(legend=dict(
    yanchor="bottom",
    y=0.20,
    xanchor="right",
    x=0.99
))
fig.show()

# Filter the dataframe for rows with the specified element symbols
filtered_df = finalDf[finalDf["element_symbol"].isin(nonmetals)]

# Create the scatter plot
fig = px.scatter(
    filtered_df,
    x="PC1",
    y="PC2",
    width=1000,
    height=1000,
    color="element_symbol",
)


# Set marker size to constant
fig.update_traces(marker=dict(size=4))

#Update axis title and font size
fig.update_layout(
    xaxis_title="PC1 (43.5%)",  # Custom label for x-axis
    yaxis_title="PC2 (24.8%)",  # Custom label for y-axis
    xaxis=dict(
        title_font=dict(size=24),  # Increase font size of x-axis title
        tickfont=dict(size=15),    # Increase font size of x-axis tick labels
    ),
```

```
  yaxis=dict(
    title_font=dict(size=24),  # Increase font size of y-axis title
    tickfont=dict(size=15),    # Increase font size of y-axis tick labels
  ),
  legend=dict(
    title=dict(font=dict(size=22)),  # Font size for the legend title
    font=dict(size=16),           # Font size for the legend items
)
)

# Update the axis lines (spines)
fig.update_layout(
  xaxis=dict(
    showline=True,         # Show bottom line (x-axis)
    linewidth=2,           # Set line width for the bottom line
    linecolor="black",      # Set line color for the bottom line
    ticks="outside",       # Place ticks outside of the axis
    tickwidth=2,           # Set tick width
    showgrid=False,        # Hide gridlines on the x-axis
  ),
  yaxis=dict(
    showline=True,         # Show left line (y-axis)
    linewidth=2,           # Set line width for the left line
    linecolor="black",      # Set line color for the left line
    ticks="outside",       # Place ticks outside of the axis
    tickwidth=2,           # Set tick width
    showgrid=False,        # Hide gridlines on the y-axis
  ),
  plot_bgcolor='white'        # Set the background color to white
)

# Add zero lines
fig.update_xaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')
fig.update_yaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')

# Adjust annotation positioning to avoid overlap
for i, feature in enumerate(features):
  # Set individual shifts for each feature
  if feature == "energy_n_minus_one":
    xshift, yshift = 72, -15 # Example shifts for feature1
  elif feature == "energy_relative":
    xshift, yshift = 55, -8  # Example shifts for feature2
  elif feature == "energy_n_plus_one":
    xshift, yshift = 70, 0  # Example shifts for feature3
  elif feature == "n_val_electrons":
    xshift, yshift = -15, 10  # Example shifts for feature3
  elif feature == "magnetic_moment":
    xshift, yshift = -1, 0 # Example shifts for feature3
```

```
    elif feature == "homo_lumo_gap":
        xshift, yshift = 0, -13  # Example shifts for feature3
    else:
        xshift, yshift = 30, -13    # Default shifts for others

    # Add arrows for the loadings
    fig.add_annotation(
        ax=0, ay=0,
        axref="x", ayref="y",
        x=loadings[i, 0],
        y=loadings[i, 1],
        showarrow=True,
        arrowsize=2,
        arrowhead=1,
        xanchor="left",
        yanchor="bottom",
        arrowcolor="black"  # Change to desired color for the arrow
    )

    # Add text annotations with individual shifts
    fig.add_annotation(
        x=loadings[i, 0],
        y=loadings[i, 1],
        ax=0, ay=0,
        xanchor="center",  # Adjust horizontal anchor
        yanchor="bottom",   # Adjust vertical anchor
        text=f"<b>{feature}</b>",
        yshift=yshift,    # Individual y shift
        xshift=xshift ,    # Individual x shift
        font=dict(
        color="black",    # Set text color
    ),

    )
fig.update_layout(legend=dict(
    yanchor="bottom",
    y=0.20,
    xanchor="right",
    x=0.99
))

fig.show()

# Filter the dataframe for rows with the specified element symbols
filtered_df = finalDf[finalDf["element_symbol"].isin(["Y", "Sc", "Ni"])]

# Create the scatter plot
fig = px.scatter(
```

```
    filtered_df,
    x="PC1",
    y="PC2",
    width=1000,
    height=1000,
    color="element_symbol",
    size="n_atoms",
)

#Update axis title and font size
fig.update_layout(
    xaxis_title="PC1 (43.5%)",  # Custom label for x-axis
    yaxis_title="PC2 (24.8%)",  # Custom label for y-axis
    xaxis=dict(
        title_font=dict(size=24),  # Increase font size of x-axis title
        tickfont=dict(size=15),    # Increase font size of x-axis tick labels
    ),
    yaxis=dict(
        title_font=dict(size=24),  # Increase font size of y-axis title
        tickfont=dict(size=15),    # Increase font size of y-axis tick labels
    ),
    legend=dict(
        title=dict(font=dict(size=22)),  # Font size for the legend title
        font=dict(size=16),           # Font size for the legend items
)
)

# Update the axis lines (spines)
fig.update_layout(
    xaxis=dict(
        showline=True,         # Show bottom line (x-axis)
        linewidth=2,           # Set line width for the bottom line
        linecolor="black",      # Set line color for the bottom line
        ticks="outside",       # Place ticks outside of the axis
        tickwidth=2,           # Set tick width
        showgrid=False,        # Hide gridlines on the x-axis
    ),
    yaxis=dict(
        showline=True,         # Show left line (y-axis)
        linewidth=2,           # Set line width for the left line
        linecolor="black",      # Set line color for the left line
        ticks="outside",       # Place ticks outside of the axis
        tickwidth=2,           # Set tick width
        showgrid=False,        # Hide gridlines on the y-axis
    ),
    plot_bgcolor='white'        # Set the background color to white
)
```

```
# Add zero lines
fig.update_xaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')
fig.update_yaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')

# Adjust annotation positioning to avoid overlap
for i, feature in enumerate(features):
    # Set individual shifts for each feature
    if feature == "energy_n_minus_one":
        xshift, yshift = 72, -15 # Example shifts for feature1
    elif feature == "energy_relative":
        xshift, yshift = 55, -8  # Example shifts for feature2
    elif feature == "energy_n_plus_one":
        xshift, yshift = 70, 0  # Example shifts for feature3
    elif feature == "n_val_electrons":
        xshift, yshift = -15, 10  # Example shifts for feature3
    elif feature == "magnetic_moment":
        xshift, yshift = -1, 0 # Example shifts for feature3
    elif feature == "homo_lumo_gap":
        xshift, yshift = 0, -13  # Example shifts for feature3
    else:
        xshift, yshift = 30, -13    # Default shifts for others

    # Add arrows for the loadings
    fig.add_annotation(
        ax=0, ay=0,
        axref="x", ayref="y",
        x=loadings[i, 0],
        y=loadings[i, 1],
        showarrow=True,
        arrowsize=2,
        arrowhead=1,
        xanchor="left",
        yanchor="bottom",
        arrowcolor="black"  # Change to desired color for the arrow
    )

    # Add text annotations with individual shifts
    fig.add_annotation(
        x=loadings[i, 0],
        y=loadings[i, 1],
        ax=0, ay=0,
        xanchor="center",  # Adjust horizontal anchor
        yanchor="bottom",   # Adjust vertical anchor
        text=f"<b>{feature}</b>",
        yshift=yshift,     # Individual y shift
        xshift=xshift ,     # Individual x shift
        font=dict(
        color="black",    # Set text color
```

```
  ),

  )
fig.update_layout(legend=dict(
  yanchor="bottom",
  y=0.20,
  xanchor="right",
  x=0.99
))

fig.show()

# Create the scatter plot
fig = px.scatter(
  finalDf,
  x="PC1",
  y="PC2",
  width=1000,
  height=1000,
  color="element_symbol",
  size="n_atoms",
)

#Update axis title and font size
fig.update_layout(
  xaxis_title="PC1 (43.5%)",  # Custom label for x-axis
  yaxis_title="PC2 (24.8%)",  # Custom label for y-axis
  xaxis=dict(
    title_font=dict(size=24),  # Increase font size of x-axis title
    tickfont=dict(size=15),    # Increase font size of x-axis tick labels
  ),
  yaxis=dict(
    title_font=dict(size=24),  # Increase font size of y-axis title
    tickfont=dict(size=15),    # Increase font size of y-axis tick labels
  ),
  legend=dict(
    title=dict(font=dict(size=22)),  # Font size for the legend title
    font=dict(size=16),           # Font size for the legend items
)
)

# Update the axis lines (spines)
fig.update_layout(
  xaxis=dict(
    showline=True,          # Show bottom line (x-axis)
    linewidth=2,           # Set line width for the bottom line
    linecolor="black",      # Set line color for the bottom line
    ticks="outside",        # Place ticks outside of the axis
```

```
        tickwidth=2,           # Set tick width
        showgrid=False,        # Hide gridlines on the x-axis
    ),
    yaxis=dict(
        showline=True,         # Show left line (y-axis)
        linewidth=2,           # Set line width for the left line
        linecolor="black",     # Set line color for the left line
        ticks="outside",       # Place ticks outside of the axis
        tickwidth=2,           # Set tick width
        showgrid=False,        # Hide gridlines on the y-axis
    ),
    plot_bgcolor='white'       # Set the background color to white
)

# Add zero lines
fig.update_xaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')
fig.update_yaxes(zeroline=True, zerolinewidth=2, zerolinecolor='lightgray')

# Adjust annotation positioning to avoid overlap
for i, feature in enumerate(features):
    # Set individual shifts for each feature
    if feature == "energy_n_minus_one":
        xshift, yshift = 72, -15 # Example shifts for feature1
    elif feature == "energy_relative":
        xshift, yshift = 55, -8  # Example shifts for feature2
    elif feature == "energy_n_plus_one":
        xshift, yshift = 70, 0  # Example shifts for feature3
    elif feature == "n_val_electrons":
        xshift, yshift = -15, 10  # Example shifts for feature3
    elif feature == "magnetic_moment":
        xshift, yshift = -1, 0 # Example shifts for feature3
    elif feature == "homo_lumo_gap":
        xshift, yshift = 0, -13  # Example shifts for feature3
    else:
        xshift, yshift = 30, -13    # Default shifts for others

    # Add arrows for the loadings
    fig.add_annotation(
        ax=0, ay=0,
        axref="x", ayref="y",
        x=loadings[i, 0],
        y=loadings[i, 1],
        showarrow=True,
        arrowsize=2,
        arrowhead=1,
        xanchor="left",
        yanchor="bottom",
        arrowcolor="black"  # Change to desired color for the arrow
```

```
    )

    # Add text annotations with individual shifts
    fig.add_annotation(
        x=loadings[i, 0],
        y=loadings[i, 1],
        ax=0, ay=0,
        xanchor="center",  # Adjust horizontal anchor
        yanchor="bottom",   # Adjust vertical anchor
        text=f"<b>{feature}</b>",
        yshift=yshift,     # Individual y shift
        xshift=xshift ,     # Individual x shift
        font=dict(
        color="black",    # Set text color
    ),

    )

fig.show()
```